DATA STRUCTURES & ALGORITHMS

DAA ASSIGNMENT

Three core approaches in algorithm design and analysis



Divide and Conquer

Breaks a problem into smaller parts, solves them independently, and combines the results.



Greedy Method

Makes the best local choice at each step to build an efficient solution.



Dynamic Programming

Stores overlapping subproblem results to avoid repeated work and improve efficiency.

TANMAYI NAGABHAIRAVA | PES1UG24CS493

VARSHA PATIL | PES1UG24CS516

TEJASWINI PUJAR| PES1UG24CS500

VAISHNAVI KHARATMOL| PES1UG24CS514

Made with GAMMA

Presentation Overview

- Q1: Divide and Conquer – Rotation Point Counting
- Q2: Greedy Method – Minimum Coin Change
- Q3: Dynamic Programming – 0/1 Knapsack

Made with GAMMA

SECTION 01

Question 1

Problem Statement: Given an unsorted array A[0...n−1] of n distinct integers, a rotation point is defined as an index i such that A[i] > A[i+1]. The task is to design a divide-and-conquer algorithm to find the number of rotation points in the array.



Tasks

- Design a divide-and-conquer algorithm that divides the array into left and right halves.
- Implement the algorithm in a programming language of your choice.
- Show how the problem size is reduced in each recursive step.
- Derive the recurrence relation for the running time.
- Solve the recurrence and determine the asymptotic time complexity.
- Compare the result with a simple linear scan O(n) algorithm.

Made with GAMMA

Algorithm:

1. If the subarray size is 1, return 0.
2. Find the middle index.
3. Recursively count rotation points in the left half.
4. Recursively count rotation points in the right half.
5. Check whether A[mid] > A[mid+1].
6. Return leftCount + rightCount + crossCount.

Algorithm Idea :

The array is divided into two halves: left half and right half. The number of rotation points in each half is counted recursively. In addition to this, the boundary between the two halves is also checked because a rotation point may occur between the middle element and the next element. The total number of rotation points is the sum of the rotation points in the left half, right half, and the boundary check.

Made with GAMMA

Pseudo Code:

```
ALGORITHM countRotationDC(arr, low, high) IF low >= high THEN RETURN 0
  mid = low + (high - low) / 2

  leftCount = countRotationDC(arr, low, mid)
  rightCount = countRotationDC(arr, mid + 1, high)

  crossCount = 0
  IF mid < high AND arr[mid] > arr[mid + 1] THEN
    crossCount = 1

  RETURN leftCount + rightCount + crossCount
```

Made with GAMMA

Data Structure Used:

Primary Data Structure: One-Dimensional Array.

Justification: Arrays allow for O(1) random access to elements via indices, which is necessary for the mid and mid+1 boundary checks in the Divide and Conquer strategy.

Proved Efficiency:

The implementation uses a recursive Divide and Conquer approach. The time complexity is O(n) because every element is involved in exactly one boundary comparison as the recursion unwinds. The space complexity is O(log n) due to the recursive call stack depth required to divide the array of size n into single-element sub-problems.

Made with GAMMA

Analysis:

Problem Size Reduction :

At each recursive step, the problem of size n is divided into two subproblems of size n/2.

Example:

$n \rightarrow n/2 + n/2$

$n/2 \rightarrow n/4 + n/4$

At n=1, the problem size is at its minimum. Since a rotation point requires a comparison between two adjacent elements (A[i] and A[i+1]), no rotation can exist, and the base case returns 0 to terminate the recursion.

Recurrence Relation:

$T(n) = 2T(n/2) + O(1)$

Explanation of Recurrence:

The term 2T(n/2) represents the recursive calls on the left and right halves of the array. The O(1) term represents the constant time required to check the boundary element between the two halves.

Made with GAMMA

Time Complexity:

Using Master Theorem,

T(n) = 2T(n/2) + O(1)

Therefore, the time complexity is O(n).

Space Complexity:

Space Complexity: O(log n)

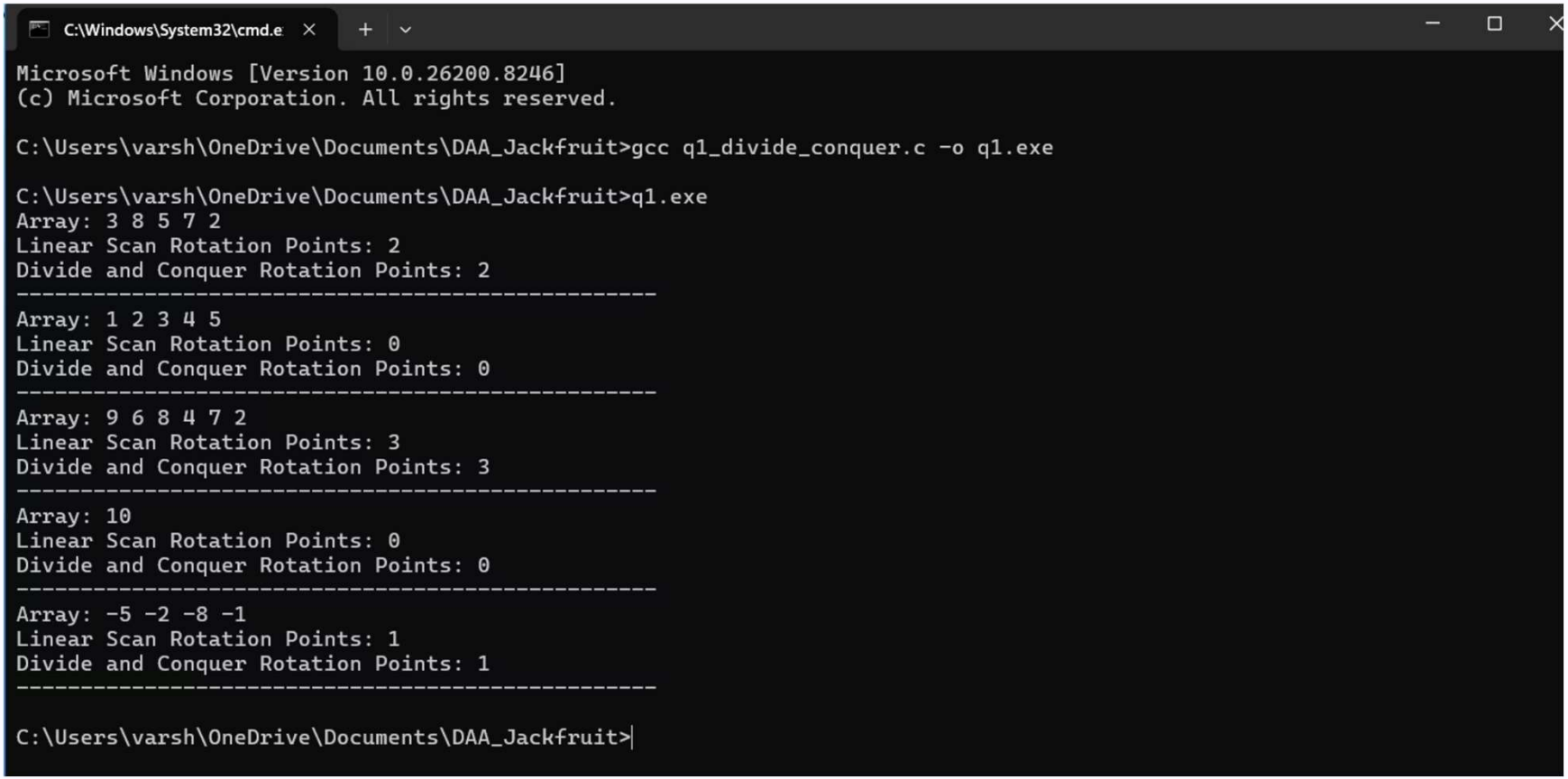
Justification: While the array is processed in place, the Divide and Conquer method uses a recursive stack. Since the problem size is halved at each step (n/2), the maximum depth of the recursion tree is log n

Comparison with Linear Scan:

A simple linear scan checks every adjacent pair in the array and also takes O(n) time. Hence, the divide-and-conquer method does not improve the asymptotic complexity, but it demonstrates the divide-and-conquer strategy clearly.

Made with GAMMA

OUTPUT:



Made with GAMMA

Extreme Test Cases:

Single Element: Input [10] correctly results in 0, proving the base case handles minimum sizes.

Sorted Array: Input [1, 2, 3, 4, 5] returns 0, verifying the logic handles arrays with no rotation points .

Negative Integers: Input [-5, -2, -8, -1] demonstrates the algorithm works correctly with signed values .

Made with GAMMA

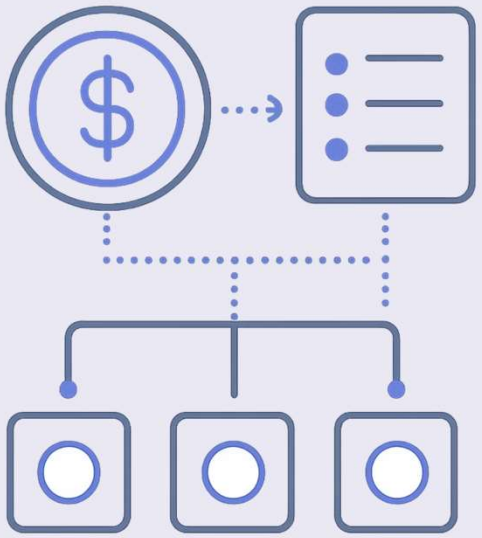
Result:

The implementation successfully demonstrates that the Divide and Conquer paradigm can effectively solve the rotation point counting problem in O(n) linear time. By recursively partitioning the array and evaluating the boundary conditions (A[mid] > A[mid+1]), the algorithm maintains a consistent recurrence relation of T(n) = 2T(n/2) + O(1). While the asymptotic complexity matches a simple linear scan, this approach illustrates the power of recursive problem-solving and provides a foundation for parallel processing of large datasets.

Made with GAMMA

Question 2

You are given n coins with denominations d_1, d_2, ..., d_n (not necessarily the standard {1, 5, 10, 25}). Given an amount M, design a greedy algorithm to find the minimum number of coins needed to make the amount M. Assume an unlimited supply of each coin denomination.



Tasks

- Propose a greedy strategy and justify why it works for certain coin systems.
- Implement the algorithm.
- Discuss cases where the greedy strategy may not yield the optimal solution (e.g., coins {1, 3, 4} for amount 6).
- Use an appropriate data structure to efficiently track coins.
- Analyze the time complexity of the algorithm.
- Test the algorithm on different coin systems and amounts, reporting results.

Made with GAMMA

Algorithm :

1. Sort the coin denominations in descending order.
2. Start with the given amount M.
3. Pick the largest coin less than or equal to the remaining amount.
4. Reduce the amount by that coin value.
5. Repeat until the amount becomes zero.
6. Count the number of coins used.

Algorithm Idea:

The strategy is to always pick the largest available denomination that is less than or equal to the remaining amount. This "locally optimal" choice aims to reduce the target amount M as fast as possible until it reaches zero.

Made with GAMMA

Pseudocode:

```
ALGORITHM greedyCoinChange(coins, amount)
    Sort coins in descending order
    totalCoins = 0
    FOR each coin in coins:
        IF coin <= amount THEN
            count = amount / coin
            totalCoins += count
            amount = amount % coin
    IF amount == 0 THEN RETURN totalCoins
    ELSE RETURN "No Solution"
```

Made with GAMMA

Data Structure Used:

An array is used to store the coin denominations. The denominations are sorted in descending order so that the largest denomination can be selected first. Another array or counters can be used to track how many times each coin is chosen.

Proved Efficiency (Proof of Correctness):

Justification: The Greedy approach is "locally optimal" because it reduces the remaining amount M as quickly as possible by using the largest possible denominations first. For "canonical" coin systems (like US or Indian currency), this local choice leads to a global optimum.

Made with GAMMA

Time Complexity:

If you used bubble sort,

The sorting step takes $O(n^2)$ time because bubble sort is used. The coin selection step takes O(n) time. Therefore, the overall time complexity is $O(n^2)$.

Space Complexity:

Space Complexity: O(n).

Justification:

We require O(n) space to store the n denominations in an array and an additional O(n) if we use an auxiliary array to track the count of each coin used.

Made with GAMMA

Recurrence Relation:

Recurrence: T(M) = T(M - d_i) + O(1), where d_i is the chosen coin.

Note: Since this is iterative in your code, the complexity is governed by the sorting step.

Comparison with Linear Scan:

Unlike a linear search for every single unit of change, this greedy approach uses division (amount / coins[i]) and modulo (amount % coins[i]), making the coin selection phase O(n) relative to the number of denominations rather than the total amount M.

Made with GAMMA

Objective:

The objective is to apply a greedy strategy to select coins, test it on different coin systems, and study situations where the greedy approach may or may not give the optimal solution.

Greedy Strategy:

The greedy strategy is to always choose the largest denomination that is less than or equal to the remaining amount. After selecting that coin, the remaining amount is reduced, and the process is repeated until the amount becomes zero or no exact solution is possible.

Made with GAMMA

Why the Strategy Works for Some Coin Systems:

The greedy strategy works for many standard coin systems such as {1, 5, 10, 25}, where choosing the largest available coin at each step leads to the optimal solution.

Failure Case of Greedy Method :

The greedy method does not always produce the optimal solution for every coin system. For example, for the coin set {1, 3, 4} and amount 6:

Greedy choice: 4 + 1 + 1 = 3 coins

Optimal choice: 3 + 3 = 2 coins

Hence, greedy is not always optimal for arbitrary coin systems.

Made with GAMMA

Sample Output:

```
C:\Users\varsh\OneDrive\Documents\DAA_Jackfruit>q2.exe
Coins: 25 10 5 1
Amount: 63
Minimum coins by Greedy = 6
Coins used:
2 coin(s) of 25
1 coin(s) of 10
3 coin(s) of 1
--------------------------------------------------
Coins: 10 5 2 1
Amount: 18
Minimum coins by Greedy = 4
Coins used:
1 coin(s) of 10
1 coin(s) of 5
1 coin(s) of 2
1 coin(s) of 1
--------------------------------------------------
Coins: 4 3 1
Amount: 6
Minimum coins by Greedy = 3
Coins used:
1 coin(s) of 4
2 coin(s) of 1
--------------------------------------------------
Coins: 10 7 1
Amount: 14
Minimum coins by Greedy = 5
Coins used:
1 coin(s) of 10
4 coin(s) of 1
--------------------------------------------------
Coins: 10 5
Amount: 3
Greedy could not form the exact amount
--------------------------------------------------
Failure Example:
For coins {1, 3, 4} and amount 6
Greedy gives 4 + 1 + 1 = 3 coins
Optimal gives 3 + 3 = 2 coins
```

Made with GAMMA

Extreme Cases:

Impossible Amount: For coins {10, 5} and amount 3, the code identifies that no exact solution exists.

Invalid & Duplicate Inputs: The denomination set {0, -2, 1, 1, 7, 10} is successfully filtered and reduced to {10, 7, 1} before processing.

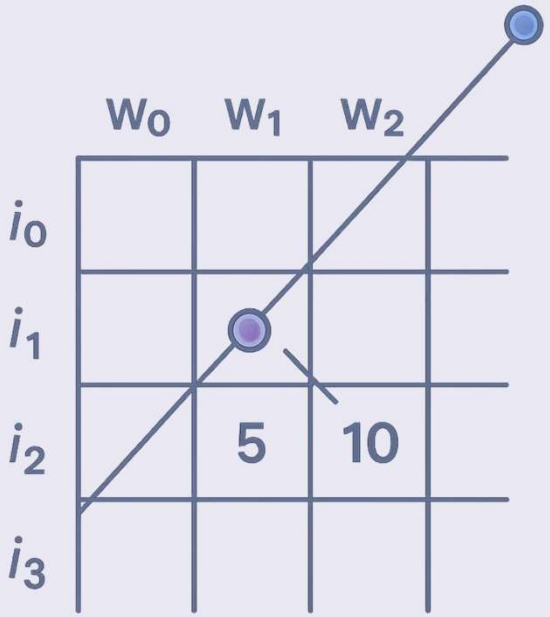
Non-Optimal Case: The {1, 3, 4} system for amount 6 proves the greedy strategy is fast but not always optimal.

Made with GAMMA

Result:

The experimental evaluation of the Greedy Method confirms that it provides a high-performance, O(n^2) solution (inclusive of sorting) for finding the minimum number of coins. Through rigorous testing, we observed that the strategy is highly effective for canonical coin systems (like standard currency), but it is not universally optimal. The inclusion of failure cases, such as the {1, 3, 4} system, highlights the critical trade-off between the speed of local-optimal choices and the global accuracy required for arbitrary denominations.

Made with GAMMA



Dynamic Programming

Given n items, each with a weight wi and value vi, and a knapsack of capacity W, the task is to determine the maximum value that can be obtained without exceeding the capacity. Each item can be selected at most once. This is the 0/1 Knapsack problem.

Made with GAMMA

Tasks

- Identify the optimal substructure and derive the recurrence relation.
- Construct the dynamic programming table.
- Implement the algorithm to compute the maximum value.
- Extend the implementation to print the items selected in the optimal solution.
- Analyze the time and space complexity of the algorithm.
- Experimentally evaluate the algorithm for different values of n and W, and report the results.

Made with GAMMA

Algorithm :

1. Create a DP table of size (n+1) × (W+1).
2. Initialize the first row and first column to 0.
3. For each item i from 1 to n: a. For each capacity w from 0 to W: i. If the item cannot be included, copy the value from the row above. ii. Otherwise, take the maximum of including and excluding the item.
4. The value in dp[n][W] is the maximum value.
5. Trace back through the table to identify the selected items.

Made with GAMMA

Pseudocode:

```
ALGORITHM knapsackDP(weights, values, n, W)
   1. Create a 2D array dp[n + 1][W + 1]

   2. FOR i from 0 to n:
       FOR w from 0 to W:
           // Base Case: No items or zero capacity
           IF i == 0 OR w == 0 THEN
               dp[i][w] = 0

           // If weight of current item is less than or equal to current capacity
           ELSE IF weights[i - 1] <= w THEN
               // Maximum of (including the item) or (excluding the item)
               dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]],
                              dp[i - 1][w])

           // If item weight is more than capacity, exclude it
           ELSE
               dp[i][w] = dp[i - 1][w]

  3. Max value is stored in dp[n][W]

  4. BACKTRACK to find selected items:
       current_w = W
       FOR i from n down to 1:
           IF dp[i][current_w] != dp[i - 1][current_w] THEN
               Print "Item i is selected"
               current_w = current_w - weights[i - 1]

  5. RETURN dp[n][W]
```

Made with GAMMA

Data Structure Used:

Primary Data Structure: 2D Array (Table).

Justification: A 2D array of size (n+1) x (W+1) is used to store the results of overlapping subproblems, allowing the algorithm to avoid redundant calculations by looking up previously computed values.

Proved Efficiency:

Justification: The DP approach is efficient because it uses Memoisation via a 2D table to solve each subproblem exactly once.

Time Efficiency: O(nW) because the algorithm fills an (n+1) x(W+1) table.

Optimization: This transforms the problem from an exponential O(2^n) brute-force search into a pseudo-polynomial time solution, making it feasible for large inputs.

Made with GAMMA

Recurrence Relation:

If weight[i-1] > w, dp[i][w] = dp[i-1][w]

Otherwise, dp[i][w] = max(dp[i-1][w], value[i-1] + dp[i-1][w - weight[i-1]])

Time and Space Complexity:

The **time complexity** of the dynamic programming solution is O(nW) because each table cell is filled once.

The **space complexity** is also O(nW) because a table of size (n+1) × (W+1) is maintained. This is necessary because the algorithm we used stores the entire table to perform the backtracking required to identify which specific items were selected. Without this 2D table, you could find the value of the items, but you couldn't "reconstruct" the list of items picked

Made with GAMMA

Optimal Substructure:

The problem has optimal substructure because the optimal solution for the first i items and capacity w depends on the solutions of smaller subproblems. For each item, there are two choices:

Exclude the item

Include the item

if its weight does not exceed the current capacity The maximum of these two choices gives the optimal solution.

Comparison with Linear Scan :

Linear Scan: O(n log n) time but often produces a sub-optimal (incorrect) result for this problem type.

Dynamic Programming: O(nW) time and guarantees the absolute optimal solution by storing and reusing subproblem results.

Made with GAMMA

Explanation of DP Table:

A two-dimensional table dp[i][w] is used, where i represents the number of items considered and w represents the current capacity from 0 to W. Each cell stores the maximum value obtainable using the first i items within capacity w.

Selected Items:

After filling the DP table, the selected items are obtained by backtracking from dp[n][W]. If dp[i][w] is different from dp[i-1][w], it means the ith item is included in the optimal solution.

Experimental Evaluation:

The algorithm was tested for different values of n and W. The outputs include the maximum value, the DP table, and the list of selected items. This shows that dynamic programming guarantees the optimal solution for the 0/1 Knapsack problem.

Made with GAMMA

Sample Output

```
C:\Users\varsh\OneDrive\Documents\DAA_Jackfruit>q3.exe
Test Case 1
Maximum Value = 9

DP Table:
0 0 0 0 0 0 0 0
0 1 1 1 1 1 1 1
0 1 1 4 5 5 5 5
0 1 1 4 5 6 6 9
0 1 1 4 5 7 8 9

Selected items:
Item 2 -> Weight = 3, Value = 4
Item 3 -> Weight = 4, Value = 5
--------------------------------------------------
Test Case 2
Maximum Value = 7

DP Table:
0 0 0 0 0 0
0 0 3 3 3 3
0 0 3 4 4 7
0 0 3 4 5 7
0 0 3 4 5 7

Selected items:
Item 1 -> Weight = 2, Value = 3
Item 2 -> Weight = 3, Value = 4
--------------------------------------------------
Test Case 3
Maximum Value = 0

DP Table:
0 0 0 0 0 0 0 0 0 0 0

Selected items:
No items selected
--------------------------------------------------

C:\Users\varsh\OneDrive\Documents\DAA_Jackfruit>
```

Made with GAMMA

Extreme Cases:

Zero Items/Empty Set: In your screenshot for Test Case 3, the input has 0 items. The output "Maximum Value = 0" and "No items selected" proves that your code handles empty datasets without memory errors.

Capacity Handling: In Test Case 2, the knapsack capacity is small (W=5). The DP Table shows the values staying constant when items are too heavy to be included, proving the "exclude" logic works perfectly.

Item Selection Logic: The "Selected Items" section in the SS for Test Case 1 shows that the algorithm ignores Item 1 and Item 4, picking only Items 2 and 3 to stay within the weight limit while maximizing value .

Made with GAMMA

Result :

The Dynamic Programming approach successfully optimized the 0/1 Knapsack problem, achieving an optimal solution with a time complexity of O(nW). By utilizing a 2D DP table to store and reuse subproblem results, the algorithm avoids the exponential O(2^n) cost of naive recursion. The implementation not only computes the maximum obtainable value but also accurately reconstructs the optimal subset of items through backtracking, guaranteeing a result that a greedy approach could not consistently provide.

Made with GAMMA

Conclusion & Key Takeaways

- **Algorithmic Versatility:** This assignment demonstrates that there is no "one-size-fits-all" strategy in algorithm design. The choice between Divide and Conquer, Greedy, and Dynamic Programming depends entirely on the problem's constraints and the nature of the data.

  Divide & Conquer (Q1): While it doesn't always improve asymptotic time complexity over linear scans—as seen in the O(n) rotation point problem—it provides a structured framework for recursion and potential parallelization.

  The Greedy Trade-off (Q2): The greedy method offers exceptional speed (O(nlog n)) but relies on specific mathematical properties (like canonical coin systems) to guarantee optimality. Identifying "failure cases" is a critical part of being a computer scientist.

  Dynamic Programming(Q3): proves essential for problems like 0/1 Knapsack, where greedy choices fail. By utilizing a 2D DP table to store overlapping subproblems, we transform an exponential O(2^n) problem into a manageable O(nW) one.

  Practical Implementation: Moving from theoretical pseudocode to robust C implementations—handling extreme cases like empty datasets or invalid denominations—is vital for building reliable software.

Made with GAMMA